

Bài 5: Robustness Analysis — Cầu nối từ Use Case đến Thiết kế

Nội dung chính

Bài này tập trung vào Robustness Analysis, một kỹ thuật trung tâm của ICONIX Process giúp nhóm phát triển bắc cầu từ mô tả hành vi (use case) sang thiết kế hướng đối tượng chi tiết một cách có hệ thống. Robustness Analysis dựa trên việc xây dựng robustness diagram, một dạng biểu đồ lai giữa class diagram và activity diagram, nhằm ràng buộc từng bước trong use case với các đối tượng trong hệ thống (boundary, controller, entity). Đây là công cụ đơn giản nhưng cực kỳ mạnh, cho phép nhóm phát hiện sớm những thiếu sót logic, đối tượng còn thiếu, kịch bản phụ chưa được mô tả, cũng như kiểm tra tính nhất quán giữa use case và domain model. Qua đó, nó giúp loại bỏ sự mơ hồ trước khi đi vào thiết kế chi tiết và lập trình

Mục tiêu cần đạt

Sau khi học xong bài này, sinh viên có thể:

- Hiểu được bối cảnh và vai trò của Robustness Analysis trong ICONIX Process, đặc biệt là tại giai đoạn chuyển tiếp từ yêu cầu sang thiết kế.
- Nắm được cấu trúc và ký hiệu cơ bản của sơ đồ robustness (boundary, controller, entity).
- Thực hiện được quy trình vẽ sơ đồ robustness cho một use case cụ thể.
- Nhận diện được các lỗi và thiếu sót trong use case thông qua phân tích robustness.
- Hiểu mối liên hệ giữa robustness diagram, domain model và sequence diagram.
- Áp dụng vào ví dụ thực tế — Internet Bookstore — để rèn luyện kỹ năng phân tích.

5.1 Bối cảnh và vấn đề thực tiễn

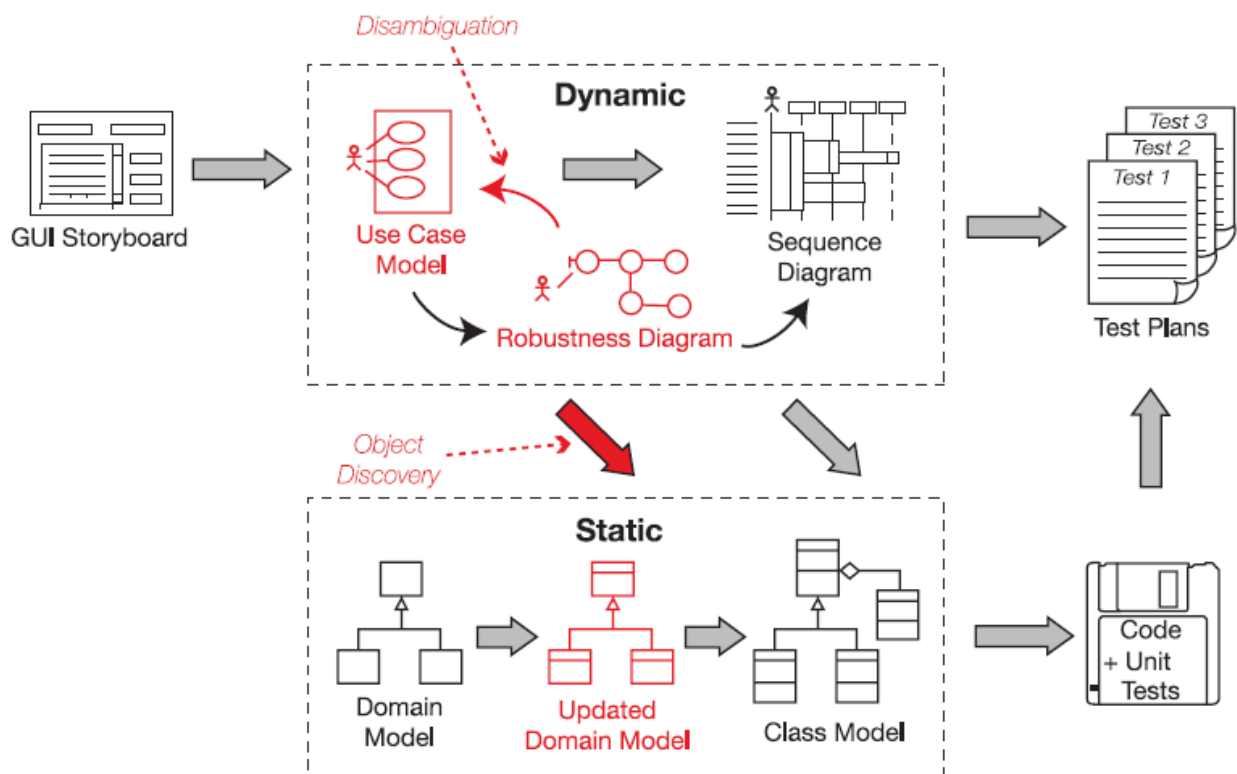
Trong thực tế phát triển phần mềm, một khoảng trống nguy hiểm luôn tồn tại giữa giai đoạn phân tích yêu cầu và giai đoạn thiết kế chi tiết.

- Ở phía trước, chúng ta có use case: mô tả hành vi của hệ thống dưới góc nhìn người dùng.
- Ở phía sau, chúng ta có thiết kế lớp và mã nguồn, nơi các hành vi được hiện thực bằng các đối tượng và phương thức cụ thể. Ở giữa hai điểm này là một vùng “xám” đầy quyết định quan trọng: ta sẽ phân rã use case thành các lớp như thế nào? Ai chịu trách nhiệm xử lý? Đối tượng nào giao tiếp với ai?

Nhiều nhóm phát triển thường nhảy thẳng từ use case sang code, vì cho rằng việc vẽ thêm sơ đồ sẽ tốn thời gian. Kết quả là:

- Mỗi lập trình viên tự “dịch” use case theo cách hiểu riêng → thiết kế bị phân mảnh, không thống nhất.
- Các lớp nghiệp vụ được tạo ra ngẫu hứng, thiếu ràng buộc chặt chẽ với yêu cầu gốc.
- Các kịch bản ngoại lệ thường bị bỏ sót.
- Khi tích hợp, hệ thống nhanh chóng trở thành “mì spaghetti”, khó mở rộng, khó kiểm thử.

Một cách tiếp cận khác là vẽ đầy đủ toàn bộ biểu đồ UML theo sách giáo khoa (class diagram, sequence, collaboration, activity...) nhưng điều này lại quá nặng và tốn công đối với hầu hết các dự án thực tế. Chính vì vậy, ICONIX Process đề xuất một bước trung gian gọn nhẹ nhưng cực kỳ hiệu quả: Robustness Analysis. Đây là bước “lọc tạp chất” trước khi đúc khuôn thiết kế. Nó giúp kiểm tra logic, phát hiện thiếu sót, và xây dựng một thiết kế sơ khai rõ ràng mà không sa vào chi tiết thừa.



5.2 Vị trí của Robustness Analysis trong ICONIX Process

Robustness Analysis nằm trong giai đoạn Phân tích & Thiết kế sơ bộ của ICONIX, ngay sau khi:

- Domain model đã được xây dựng ở mức đầu tiên.

- Các use case đã được viết rõ ràng (gồm luồng chính và luồng phụ).
- Storyboard giao diện đã được phác thảo.

Đây cũng chính là giai đoạn diễn ra Preliminary Design Review (PDR) — Cột mốc kiểm duyệt số 2 của ICONIX.

Trong buổi PDR, nhóm sẽ rà soát:

- Mỗi use case đã có sơ đồ robustness tương ứng chưa.
- Các entity và boundary trên sơ đồ có khớp với domain model không.
- Các luồng xử lý đã bao phủ cả luồng chính và phụ.
- Không có “khoảng trống logic” giữa hành vi và cấu trúc

Kết quả của bước này là một bộ sơ đồ robustness được duyệt, sẵn sàng làm đầu vào cho sequence diagram ở giai đoạn thiết kế chi tiết.

Nếu bỏ qua bước này, sequence diagram rất dễ trở nên rời rạc hoặc phải chỉnh sửa lớn sau này.

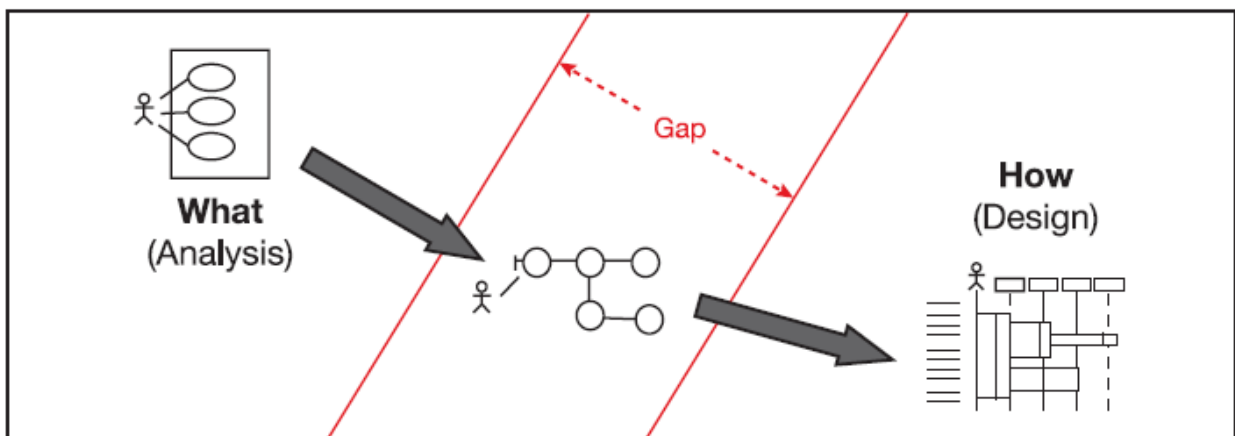


Figure 5-1. Bridging the gap between “what” and “how”

5.3 Khái niệm và cấu trúc sơ đồ robustness

Một **robustness diagram** có thể hiểu đơn giản là “**một bức tranh đối tượng của use case**”.

Nó minh họa rõ ràng:

- Ai (actor) tương tác với hệ thống qua đâu (boundary)?
- Hệ thống thực hiện logic gì (controller)?
- Đối tượng miền nào được truy xuất hoặc thay đổi (entity)?
- Dòng thông điệp nào chảy giữa các thành phần này?

Ba loại đối tượng chính

Loại	Vai trò	Ví dụ Internet Bookstore
Boundary	Giao tiếp với tác nhân bên ngoài, thường là màn hình, form, trang web hoặc API.	LoginPage, BookDetailsPage
Controller	“Chất keo” xử lý logic, điều phối luồng tương tác giữa boundary ↔ entity.	AuthenticateUser, DisplayBookDetails
Entity	Các lớp miền phản ánh thực thể nghiệp vụ, lấy từ domain model.	UserAccount, Book, ShoppingCart

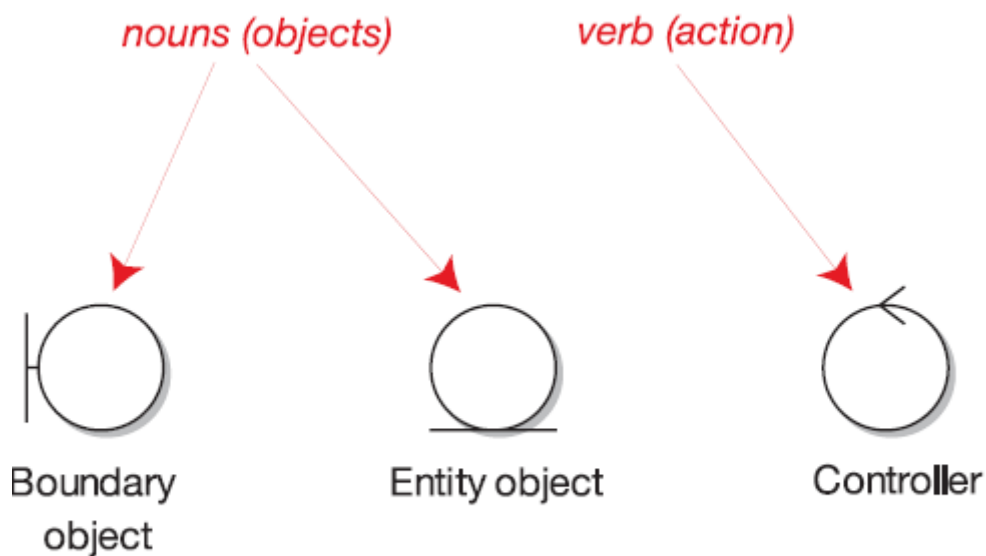


Figure 5-2. *Robustness diagram symbols*

Quy tắc giao tiếp (Noun–Verb–Noun)

- Boundary ↔ Controller
- Controller ↔ Entity
- Controller ↔ Controller
- ~~Boundary ↔ Boundary~~
- ~~Entity ↔ Entity~~
- ~~Boundary ↔ Entity~~

Quy tắc này phản ánh đúng bản chất tương tác trong một hệ thống hướng đối tượng:

Người dùng (boundary) kích hoạt hành vi (controller), hành vi này truy xuất hoặc thao tác trên dữ liệu (entity). Nếu sơ đồ của bạn vi phạm quy tắc này (ví dụ boundary nói

chuyện trực tiếp với entity), đó là một “tín hiệu cảnh báo” rằng thiết kế hoặc use case của bạn có vấn đề.

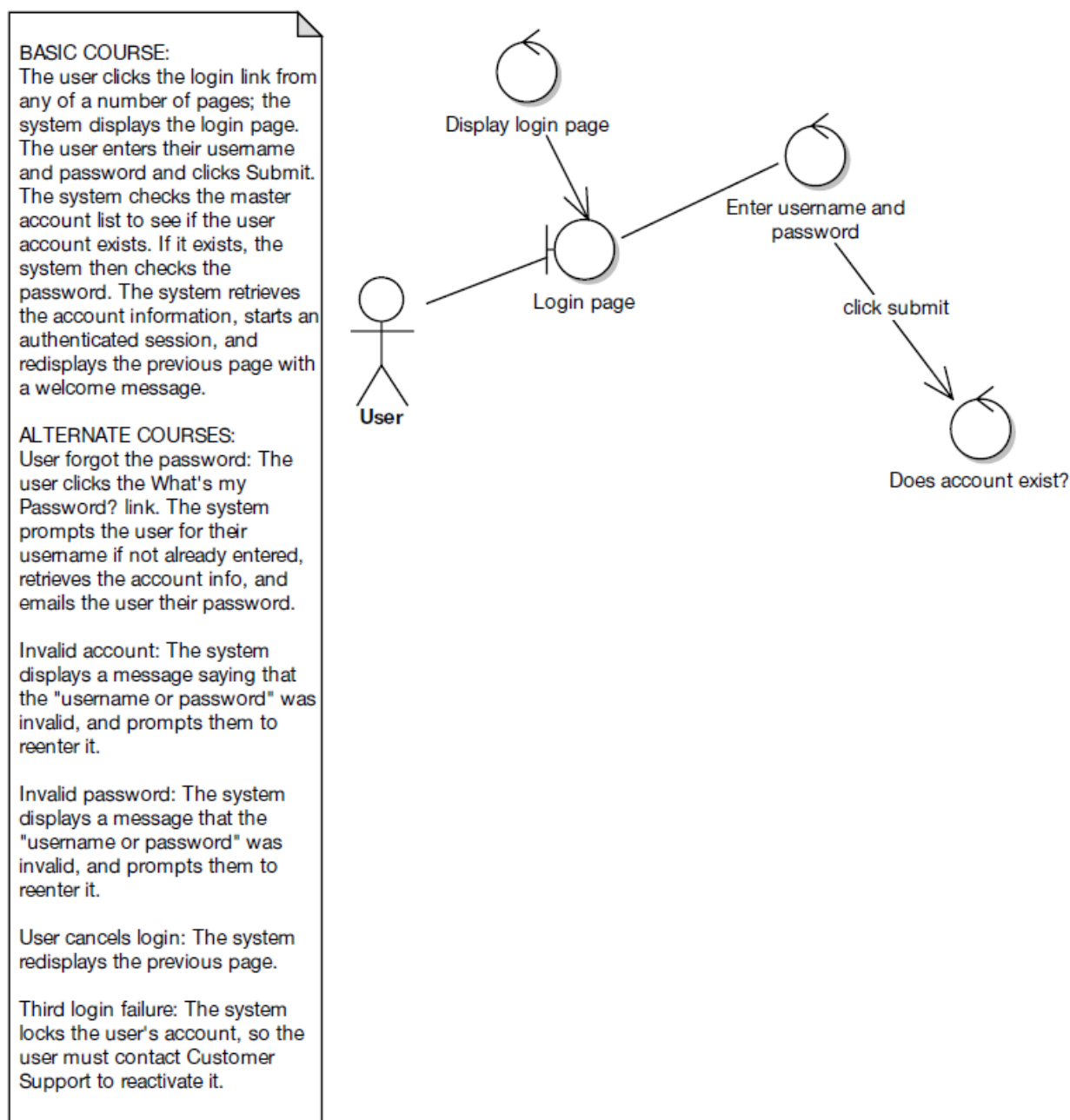


Figure 5-3. *Partially completed robustness diagram with the use case text pasted in*

5.4 Quy trình thực hiện Robustness Analysis

Khi mới học, nhiều sinh viên có xu hướng nghĩ rằng vẽ một sơ đồ robustness chỉ là “đặt vài hình tròn và mũi tên” cho đẹp. Nhưng thực chất, đây là một kỹ thuật tư duy hệ thống rất mạnh, được thiết kế để ép nhóm phát triển làm rõ từng câu chữ trong use case và ràng buộc nó với các đối tượng cụ thể trong mô hình. Chính vì vậy, ICONIX nhấn mạnh rằng: nếu bạn không thể vẽ một sơ đồ robustness đơn giản cho một use case, thì bạn chưa thực sự hiểu use case đó.

Quy trình thực hiện Robustness Analysis cho một use case có thể chia thành 6 bước sau:

Bước 1: Chọn một use case cụ thể

Mỗi use case cần một sơ đồ robustness riêng, vì mỗi use case có hành vi và phạm vi khác nhau. Đừng cố gắng “gom nhiều use case vào một sơ đồ” — điều này sẽ làm mất tác dụng kiểm soát logic của phương pháp.

Ở bước này, hãy chọn một use case có độ phức tạp vừa phải để nhóm luyện tập (ví dụ: “Login”, “Show Book Details”). Khi đã thành thạo, các use case phức tạp hơn như “Write Customer Review” sẽ trở nên dễ dàng hơn nhiều.

Bước 2: Dán toàn bộ use case text lên sơ đồ

Đây là một mẹo thực hành rất đặc trưng của ICONIX:

“Paste the use case text directly onto your robustness diagram.”

Mục đích không phải để trang trí mà là để ép bạn đọc từng câu trong use case, và vẽ từng hành động tương ứng lên sơ đồ. Nhờ vậy, bạn sẽ không bỏ sót bất kỳ bước nào trong luồng chính và luồng phụ.

Hình 5-3 minh họa một ví dụ sơ đồ robustness đang trong quá trình xây dựng cho Internet Bookstore, dành cho use case Login. Đây là một ảnh chụp của sơ đồ ở giai đoạn ban đầu; cho đến thời điểm này, mới chỉ có một vài câu đầu tiên của use case được thể hiện trên sơ đồ. Trong Hình 5-3, văn bản use case đã được dán trực tiếp vào một ghi chú trên sơ đồ. Vì sơ đồ robustness về bản chất là một hình ảnh thể hiện use case, nên việc đặt văn bản ngay trong sơ đồ giúp ích rất nhiều: đây là hai góc nhìn khác nhau của cùng một nội dung, nên bạn có thể đi theo phần văn bản để lần theo sơ đồ (và ngược lại).

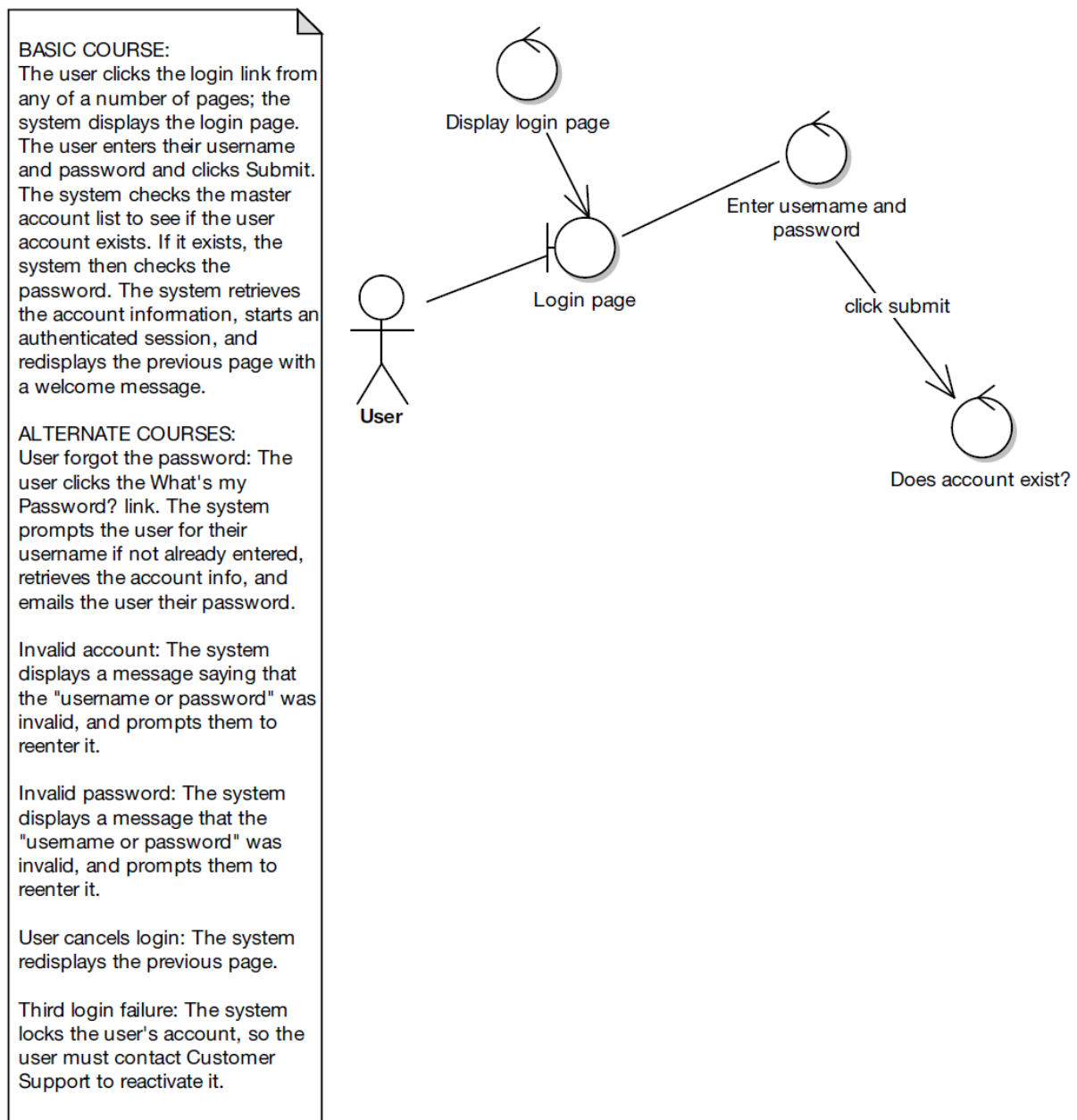


Figure 5-3. *Partially completed robustness diagram with the use case text pasted in*

Bước 3: Xác định các đối tượng tham gia

Khi đọc từng câu, hãy gạch chân:

- Các danh từ → thường là boundary hoặc entity.
- Các động từ hoặc cụm hành động → thường là controller.

Từ đó, bạn xây dựng danh sách sơ bộ các đối tượng sẽ có mặt trong sơ đồ:

- Boundary: các màn hình, trang web, form... mà người dùng tương tác.
- Entity: các lớp miền có liên quan đến hành động.

- Controller: các hành động xử lý logic (kiểm tra, hiển thị, xác thực, lưu trữ...).

Ví dụ, với câu:

“The user enters their username and password and clicks Submit.” → User (actor), LoginPage (boundary), AuthenticateUser (controller), UserAccount (entity).

Bước 4: Vẽ các mối quan hệ theo quy tắc Noun–Verb–Noun

Mỗi hành động được thể hiện bằng một mũi tên giữa các đối tượng:

- Boundary → Controller (user action)
- Controller → Entity (truy xuất hoặc cập nhật dữ liệu)
- Controller → Controller (gọi logic nội bộ nếu cần)

Quan trọng là: mỗi bước trong use case phải tương ứng với một hoặc nhiều mũi tên trong sơ đồ. Nếu bạn không thể biểu diễn một câu trên sơ đồ → hoặc use case viết chưa rõ, hoặc thiếu đối tượng.

Bước 5: Cập nhật domain model nếu phát hiện thiếu entity

ICONIX Process thừa nhận rằng domain model ban đầu luôn chưa hoàn chỉnh. Trong quá trình vẽ sơ đồ robustness, bạn có thể phát hiện ra những thực thể mới (entity) mà trước đó chưa được mô hình hóa

Ví dụ, khi phân tích use case “Write Customer Review”, bạn có thể nhận ra rằng cần thêm entity “ReviewModerationQueue” để lưu tạm các review chờ kiểm duyệt — thực thể này không xuất hiện trong domain model ban đầu. Đây gọi là Object Discovery — một bước tự nhiên và được khuyến khích trong ICONIX. Domain model được cập nhật dần theo hiểu biết mới.

Bước 6: Bao phủ cả luồng chính và luồng phụ

Một sơ đồ robustness đúng nghĩa phải thể hiện đầy đủ cả Basic Course lẫn Alternate Courses của use case. Các nhánh luồng phụ thường được vẽ bằng màu khác hoặc đặt lệch dòng chính để dễ phân biệt.

Ví dụ, trong use case “Login”, các luồng phụ gồm:

- Quên mật khẩu
- Tài khoản không tồn tại
- Mật khẩu sai
- Hủy login
- Khóa tài khoản sau 3 lần thất bại

Mỗi luồng phụ này đòi hỏi thêm controller hoặc boundary riêng (ví dụ: ForgotPasswordController, ErrorMessagePage...). Việc đưa chúng lên cùng sơ đồ giúp đảm bảo không có kịch bản nào bị bỏ sót trước khi thiết kế chi tiết.

5.5 Top 10 Guidelines — Nguyên tắc vàng khi phân tích robustness

Tác giả ICONIX đã tổng kết quá trình hàng trăm dự án thực tế thành 10 nguyên tắc rất dễ nhớ nhưng cực kỳ hiệu quả. Dưới đây là bản chuyển ngữ có giải thích ngắn:

#	Guideline	Ý nghĩa thực tế
10	Paste use case text vào sơ đồ	Đảm bảo bám sát từng câu chữ, không sót bước
9	Lấy entity từ domain model, thêm nếu thiếu	Giữ tính nhất quán và khám phá đối tượng mới
8	Kỳ vọng phải viết lại use case	Phát hiện mơ hồ khi vẽ sơ đồ là điều bình thường
7	Boundary rõ ràng, tên màn hình không mơ hồ	Tránh dùng từ chung chung như “Web page”
6	Controller chủ yếu là hàm logic, không phải lúc nào cũng là class	Giữ sơ đồ ở mức khái niệm
5	Không quan trọng hướng mũi tên	Đây không phải sequence diagram
4	Có thể kéo use case con vào sơ đồ cha	Cho phép mô tả sự gọi lồng nhau
3	Đây là conceptual design, không phải detailed design	Giữ đúng mức trừu tượng
2	Boundary & Entity → object; Controller → message trong sequence	Chuẩn bị cho bước tiếp theo
1	Sơ đồ phải phản ánh toàn bộ use case (basic + alternate)	Đây là mục tiêu chính của robustness analysis

Khi bạn thực hiện đúng 10 nguyên tắc này, sơ đồ robustness sẽ trở thành “bản thiết kế sơ khai” cực kỳ mạnh mẽ: vừa đủ đơn giản để nhanh, vừa đủ chặt chẽ để phát hiện lỗi.

5.6 Ví dụ minh họa chi tiết

ICONIX luôn dùng hệ thống **Internet Bookstore** làm ví dụ chuẩn. Ta sẽ đi qua 3 use case tiêu biểu:

Ví dụ 1: Login Use Case

Use case:

- Người dùng nhấn vào link “Login” từ bất kỳ trang nào → hệ thống hiển thị màn hình đăng nhập.

- Người dùng nhập username/password → nhấn Submit.
- Hệ thống kiểm tra tài khoản và mật khẩu → nếu đúng, tạo phiên làm việc và trở về trang trước với lời chào.
- Nếu quên mật khẩu → gửi email.
- Nếu nhập sai → hiển thị lỗi.
- Sau 3 lần sai → khóa tài khoản.

Boundary:

- LoginPage
- ErrorPage
- ForgotPasswordPage

Controller:

- DisplayLoginPage
- AuthenticateUser
- SendPasswordEmail
- LockAccount

Entity:

- UserAccount

Sơ đồ robustness cho thấy rõ mạch:

LoginPage → AuthenticateUser → UserAccount → (success/failure) → hiển thị lại trang hoặc chuyển hướng.

Các alternate như “Forgot Password” thêm controller riêng.

Điều thú vị là khi vẽ, bạn sẽ phát hiện rằng “Forgot Password” thực ra không phải là một flow nhỏ, mà là một use case khác được gọi lồng vào, nên có thể kéo nó như một “invoked use case” vào sơ đồ cha

BASIC COURSE:

The user clicks the login link from any of a number of pages; the system displays the login page. The user enters their username and password and clicks Submit. The system checks the master account list to see if the user account exists. If it exists, the system then checks the password. The system retrieves the account information, starts an authenticated session, and redisplay the previous page with a welcome message.

ALTERNATE COURSES:

User forgot the password: The user clicks the What's my Password? link. The system prompts the user for their username if not already entered, retrieves the account info, and emails the user their password.

Invalid account: The system displays a message saying that the "username or password" was invalid, and prompts them to reenter it.

Invalid password: The system displays a message that the "username or password" was invalid, and prompts them to reenter it.

User cancels login: The system redisplay the previous page.

Third login failure: The system locks the user's account, so the user must contact Customer Support to reactivate it.

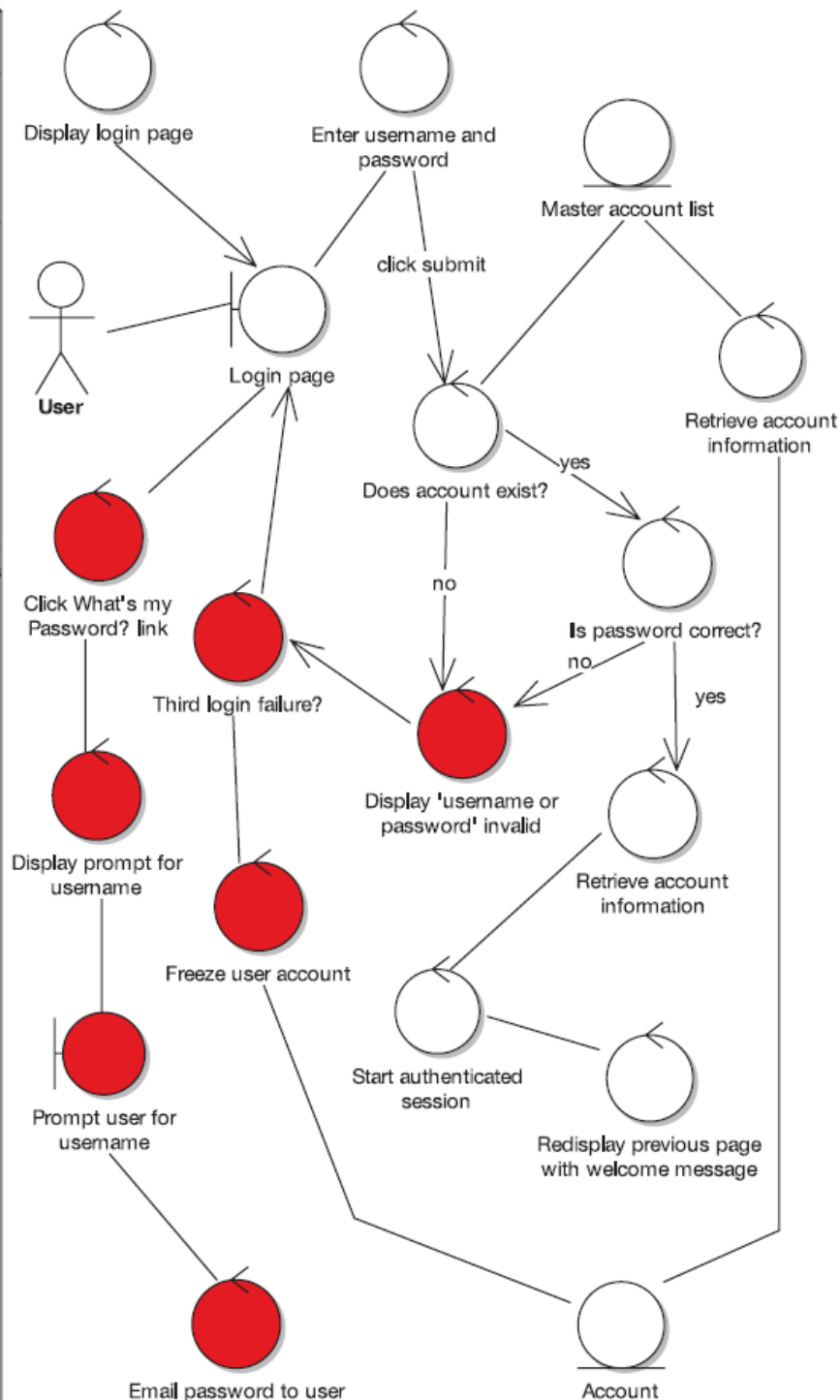


Figure 5-5. Example robustness diagram

Ví dụ 2: Show Book Details Use Case

Đây là một use case đơn giản nhưng lại minh họa rất rõ sức mạnh của Robustness Analysis trong việc làm sáng tỏ các đối tượng, màn hình, và luồng điều khiển

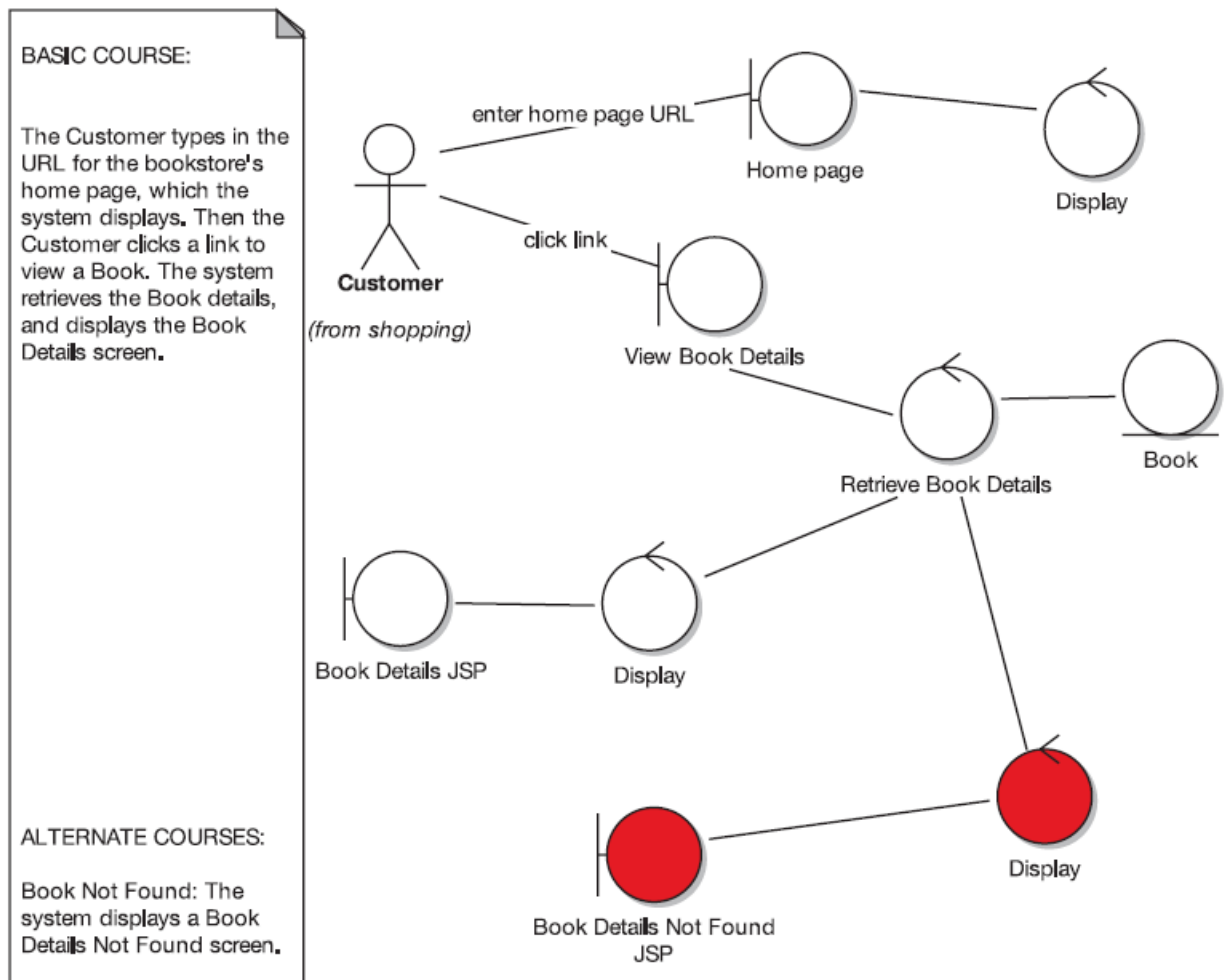


Figure 5-10. A first attempt at a robustness diagram for the Show Book Details use case

Mô tả use case (Basic course):

1. Người dùng đang ở trang chủ (Home Page).
2. Người dùng nhấn vào liên kết của một cuốn sách bất kỳ.
3. Hệ thống truy xuất thông tin sách và hiển thị trang chi tiết sách.

Luồng phụ (Alternate):

- Cuốn sách không tồn tại → hệ thống hiển thị thông báo lỗi “Book not found”.

Phân tích đối tượng

- **Boundary:**
 - HomePage (trang nguồn nơi người dùng nhấn vào sách)
 - BookDetailsPage (trang đích hiển thị thông tin chi tiết)
 - ErrorPage (trang báo lỗi khi sách không tồn tại)

- **Controller:**
 - DisplayBookDetailsPage (điều phối từ HomePage → BookDetailsPage)
 - FetchBookDetails (lấy dữ liệu sách từ domain)
- **Entity:**
 - Book (thực thể miền chính, chứa ISBN, Title, Price, v.v.)

Phân tích quy tắc noun–verb–noun

1. **HomePage → DisplayBookDetailsPage**
Người dùng nhấn vào link sách → controller điều phối xử lý.
2. **DisplayBookDetailsPage → FetchBookDetails**
Controller chính gọi controller con để lấy dữ liệu sách.
3. **FetchBookDetails → Book**
Controller logic truy cập thực thể sách.
4. **DisplayBookDetailsPage → BookDetailsPage**
Sau khi lấy dữ liệu thành công, controller điều phối hiển thị trang chi tiết.
5. **FetchBookDetails (alternate):** Nếu không tìm thấy Book → chuyển sang ErrorPage.

Những vấn đề được làm rõ nhờ sơ đồ robustness

- Khi chưa vẽ sơ đồ, nhiều sinh viên **nhập nhằng giữa “trang view” và “controller”**: gọi “view book details page” như một boundary, trong khi thực chất phải có một controller trung gian để lấy dữ liệu và hiển thị.
- Nhiều nhóm gọi “JSP/ASP page” thẳng trong sơ đồ, khiến thiết kế trở nên phụ thuộc công nghệ → vi phạm tính khái niệm. ICONIX khuyến cáo chỉ dùng tên màn hình mang tính nghiệp vụ, **không gắn với kỹ thuật** như JSP, ASP, FXML...
- Sơ đồ cũng giúp phát hiện trường hợp alternate “book not found” dễ bị bỏ sót trong use case draft.

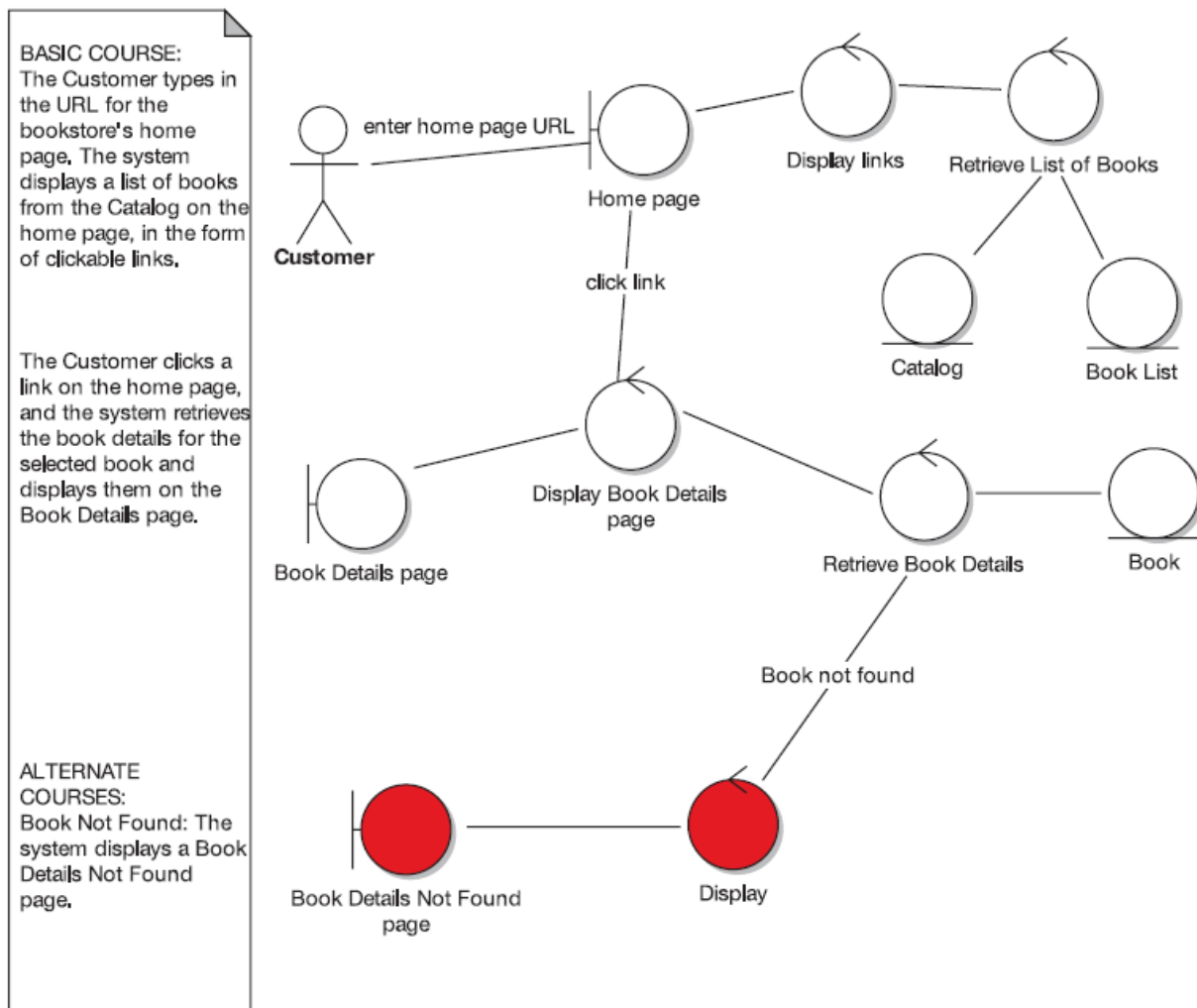


Figure 5-11. *Corrected robustness diagram for the Show Book Details use case*

Ví dụ 3: Write Customer Review Use Case

Bước 1: Tạo sơ đồ Robustness trống

“Bước đầu tiên được thể hiện trong Hình 5-12. Chúng ta tạo một sơ đồ robustness mới, còn trống.”

Câu đầu tiên trong phần Basic Course của use case:

“The Customer clicks the Write Review button for the book currently being viewed, and the system shows the Write Review screen.”

(Khách hàng nhấn nút “Write Review” cho cuốn sách hiện đang được xem, và hệ thống hiển thị màn hình “Write Review”).

Thành phần đầu tiên được nhắc tới là **Customer**, vì vậy ta cần đặt actor Customer lên sơ đồ.

Bước 2: Xác định các đối tượng giao diện ban đầu

Ở bước này, có vẻ hiển nhiên là sẽ vẽ nút “**Write Review**” như một boundary object và cho Customer tương tác với nó

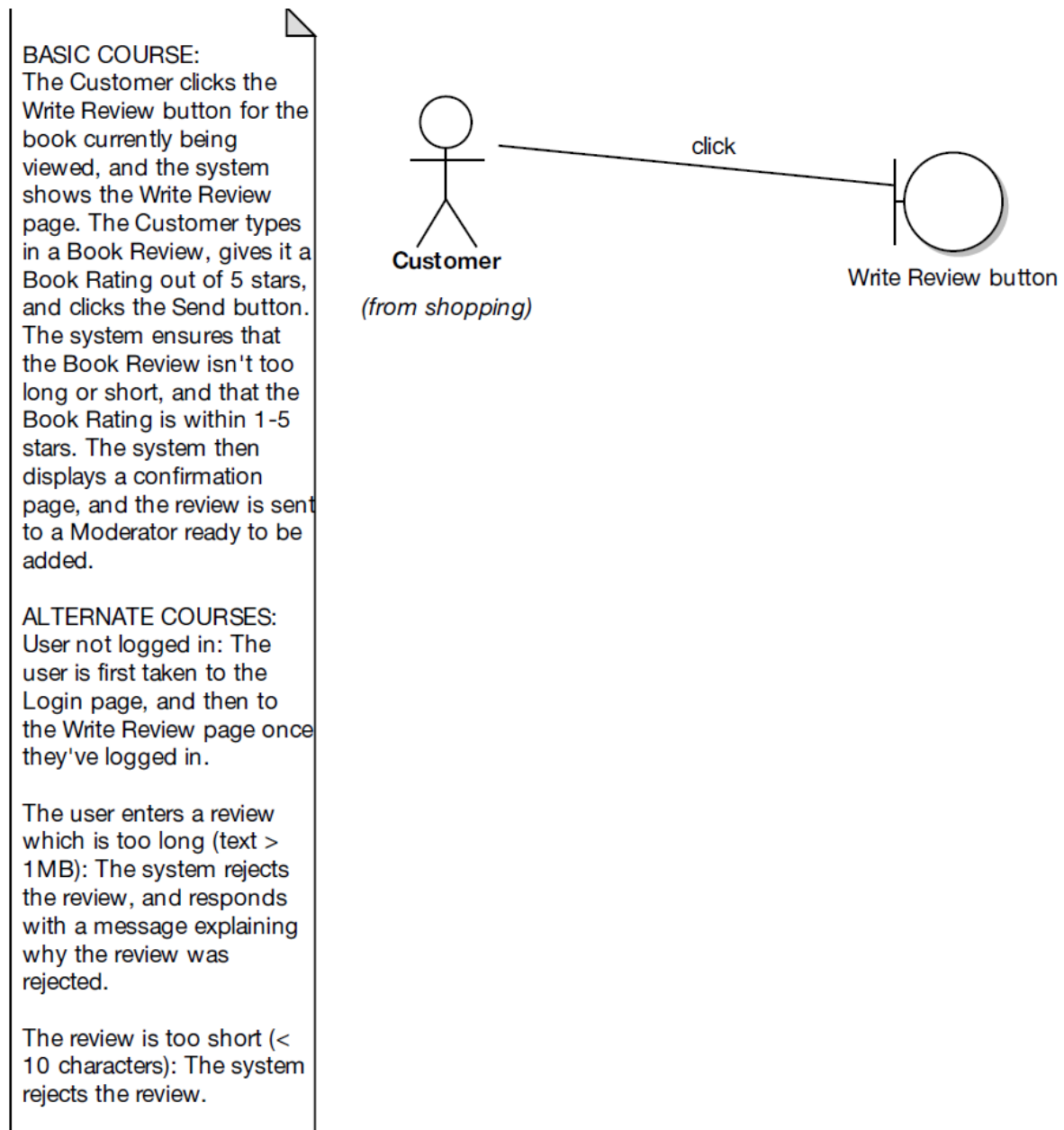


Figure 5-13. Step 2: Spot the deliberate mistake.

Tuy nhiên, nếu bạn bắt đầu đưa **các widget giao diện** (như nút, textbox...) vào sơ đồ robustness thì sẽ rất nhanh bị sa đà — bạn sẽ phải vẽ hàng đống controller và boundary cho từng field, list box, button... Vì vậy, ICONIX khuyên **không nên vẽ GUI widget cấp thấp** trong sơ đồ robustness.

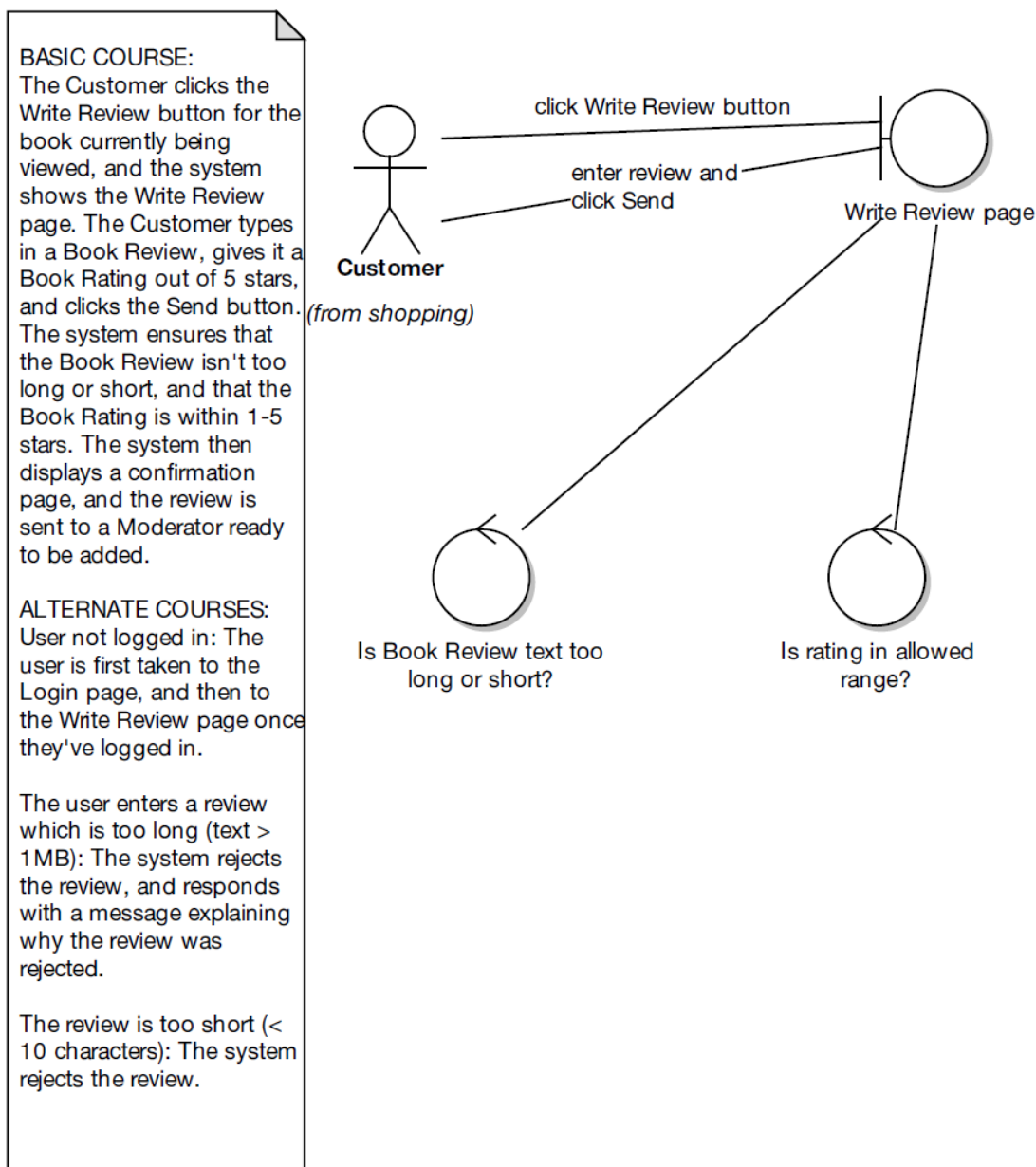


Figure 5-14. Step 3: We've now corrected the mistake and added some validation controllers.

Trong Hình 5-14, nút “Write Review Button” đã được **loại bỏ**, và thay vào đó là **một thông điệp giữa Customer và Write Review Screen**.

Basic Course của Use Case

BASIC COURSE:

The Customer clicks the Write Review button for the book currently being viewed, and the system shows the Write Review page. The Customer types in a Book Review, gives it a Book Rating out of 5 stars, and clicks the Send button. The system ensures that the Book Review isn't too long or short, and that the Book Rating is within 1–5

stars. The system then displays a confirmation page, and the review is sent to a Moderator ready to be added.

LUỒNG CHÍNH:

Khách hàng nhấn nút **Write Review** cho cuốn sách hiện đang được xem, hệ thống hiển thị trang **Write Review**. Khách hàng nhập vào nội dung đánh giá sách, chấm điểm từ 1 đến 5 sao, và nhấn nút **Send**. Hệ thống kiểm tra để đảm bảo nội dung đánh giá không quá ngắn hoặc quá dài, và điểm đánh giá nằm trong khoảng từ 1 đến 5 sao. Sau đó hệ thống hiển thị trang xác nhận, và gửi đánh giá tới Moderator để xử lý thêm.

ALTERNATE COURSES: User not logged in: The user is first taken to the Login page, and then to the Write Review page once they've logged in. The user enters a review which is too long (text > 1MB): The system rejects the review, and responds with a message explaining why the review was rejected. The review is too short (< 10 characters): The system rejects the review.

CÁC LUỒNG THAY THẾ:

Người dùng chưa đăng nhập: Hệ thống sẽ đưa người dùng tới trang **Login**, sau khi đăng nhập thành công mới chuyển tới trang **Write Review**. Người dùng nhập đánh giá quá dài (văn bản > 1MB): Hệ thống từ chối đánh giá và phản hồi bằng thông báo giải thích lý do từ chối. Người dùng nhập đánh giá quá ngắn (< 10 ký tự): Hệ thống từ chối đánh giá.

Các bước chỉnh sửa sơ đồ

Trong Hình 5-14, tác giả thêm **các controller kiểm tra độ dài review và giá trị rating** để phản ánh chính xác nội dung trong phần Basic Course:

“The Customer types in a Book Review, gives it a Book Rating out of five stars, and clicks the Send button. The system ensures that the Book Review isn't too long or short, and that the Book Rating is within one and five stars.”

Bước 4: Hoàn thiện phần còn lại của luồng chính

Hình 5-15 thể hiện đoạn văn bản tiếp theo:

“The system then displays a confirmation screen, and the review is sent to a Moderator, ready to be added.”

Trong sơ đồ, có thêm **controller Display** điều khiển việc hiển thị trang xác nhận, và **liên kết tới use case “Moderate Customer Reviews”** (thuộc gói admin) để xử lý phần gửi review cho Moderator. Tác giả cũng chỉnh lại Use Case Text để nhắc rõ **Book Detail Page**, vì câu đầu tiên đã ngụ ý điều đó.

Bước 5: Thêm luồng thay thế “User not logged in”

“User not logged in: The user is first taken to the Login screen and then to the Write Review screen once he is logged in.”

Để mô hình hóa, thêm controller “Is user logged in?”.

Nếu không đăng nhập, **invoke use case Login**; nếu đã đăng nhập, tiếp tục hiển thị trang Write Review như bình thường. Ở đây, họ cũng phát hiện ra một **đối tượng mới bị thiếu: Customer Session**, để hệ thống có thể kiểm tra trạng thái đăng nhập.

→ Khi phát hiện ra, phải **cập nhật cả Domain Model**.

Bước 6: Thêm các luồng thay thế còn lại (độ dài review không hợp lệ)

- Review quá dài (> 1MB): hệ thống từ chối và trả về thông báo.
- Review quá ngắn (< 10 ký tự): hệ thống từ chối và hiển thị thông báo lỗi.

→ Hai yêu cầu ngầm rút ra được là:

- Độ dài review không vượt quá 1MB.
- Độ dài review không dưới 10 ký tự.

Hình 5-17 minh họa sơ đồ cuối cùng sau khi thêm các luồng này.

BASIC COURSE:

On the Book Detail page for the book currently being viewed, the Customer clicks the Write Review button. The system checks the Customer Session to make sure the Customer is logged in, and then displays the Write Review page. The Customer types in a Book Review, gives it a Book Rating out of 5 stars, and clicks the Send button. The system ensures that the Book Review isn't too long or short, and that the Book Rating is within 1-5 stars. The system then displays a confirmation page, and the review is sent to a Moderator ready to be added.

ALTERNATE COURSES:

User not logged in: The user is first taken to the Login page, and then to the Write Review page once they've logged in.

The user enters a review which is too long (text > 1MB): The system rejects the review, and responds with a message explaining why the review was rejected.

The review is too short (< 10 characters): The system rejects the review.

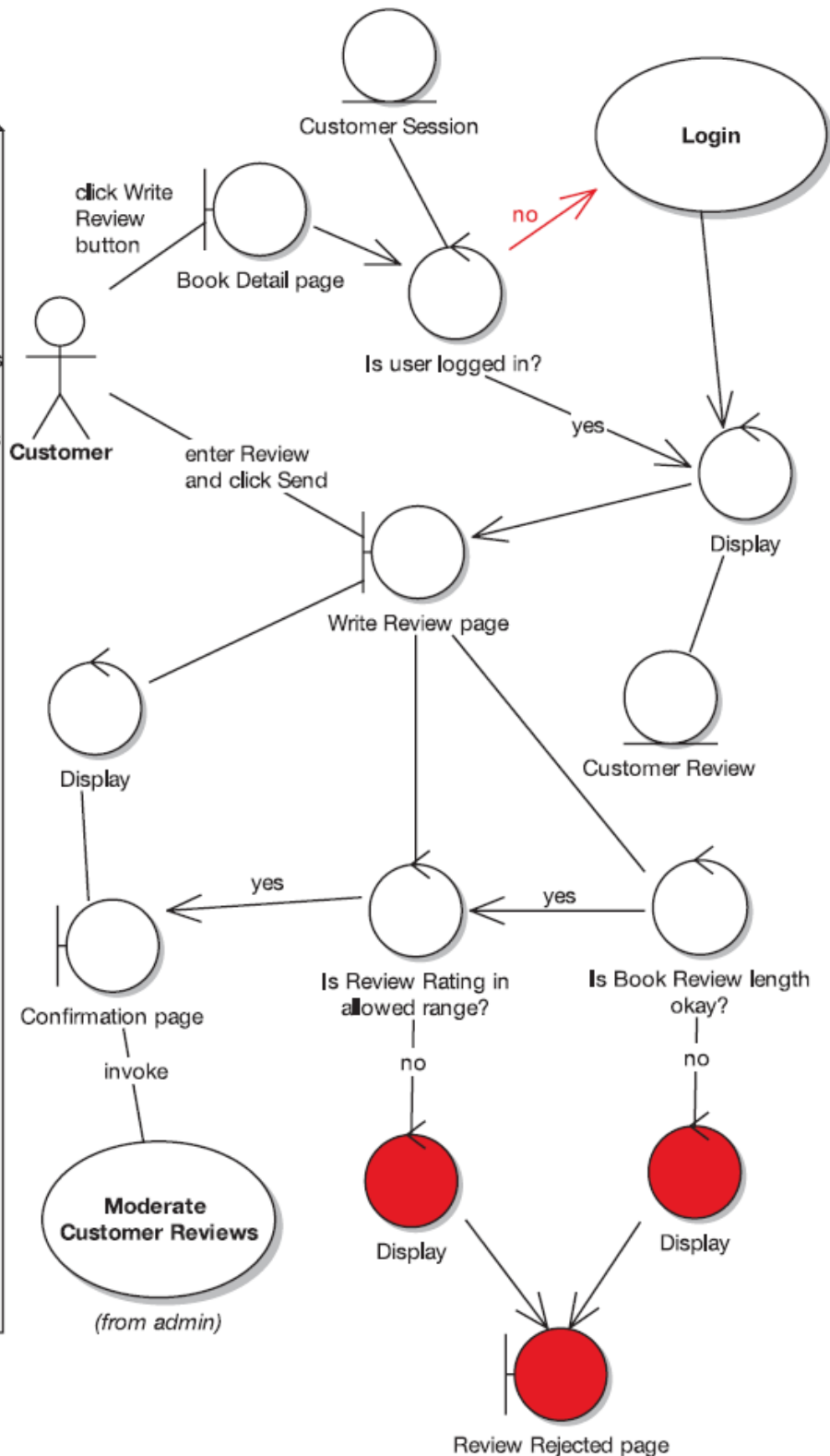


Figure 5-17. Step 6: Robustness diagram with the last two alternate courses added

5.7 Các lỗi thường gặp và cách khắc phục

Khi sinh viên hoặc nhóm dự án mới học kỹ thuật này, thường mắc phải 5 lỗi lớn sau:

Lỗi	Mô tả	Cách khắc phục
1	Không bám sát use case khi vẽ sơ đồ	Vẽ dựa theo trí nhớ, bỏ sót luồng phụ
2	Gọi sai tên boundary/controller	Lẫn lộn danh từ–động từ hoặc dùng tên kỹ thuật (JSP...)
3	Bỏ alternate flows	Chỉ vẽ basic course
4	Vẽ sơ đồ quá chi tiết như sequence diagram	Tập trung vào hướng mũi tên, thông điệp cụ thể
5	Không cập nhật domain model	Thiếu entity mới phát hiện

Khi bạn làm đúng, sơ đồ robustness trở thành một bộ lọc lỗi tự nhiên — nó buộc use case, domain model, storyboard phải “nói cùng một ngôn ngữ”.

Robustness Analysis là cầu nối quan trọng nhất trong quy trình ICONIX giữa thế giới hành vi (use case) và thế giới thiết kế (sequence diagram, class diagram).

Nó giúp:

- Ép nhóm làm rõ từng câu chữ trong use case.
- Ràng buộc logic xử lý với các đối tượng cụ thể.
- Phát hiện sớm lỗi hổng trong yêu cầu, thiếu đối tượng, luồng phụ.
- Xây dựng một thiết kế sơ khai rõ ràng, là tiền đề cho sequence diagram.

Không có bước này, bạn sẽ hoặc là “nhảy vào code” quá sớm, hoặc sa đà vào UML chi tiết mà vẫn bỏ sót kịch bản quan trọng. Có bước này, bạn chỉ cần 10 phút với bảng trắng cũng có thể khám phá những điểm mù trong phân tích mà văn bản dài không phát hiện được. Nói cách khác, Robustness Analysis chính là cây cầu bắc từ “What” đến “How” trong phát triển phần mềm hướng đối tượng

5.8 Cập nhật mô hình miền (Domain / Static Model)

Trong khi bạn đang vẽ các sơ đồ robustness, một ý tưởng rất hay là đồng thời cập nhật mô hình miền (domain model) một cách từng bước. Gần như chắc chắn rằng bạn sẽ phát hiện ra những lớp miền mới trong quá trình này. Bạn cũng sẽ xác định được các thuộc tính cần bổ sung cho những lớp đã có. Tất cả những thay đổi này phải được phản ánh vào mô hình miền (hay còn gọi là *analysis-level static model*) — và tốt nhất là nên làm ngay tại thời điểm bạn phát hiện ra chúng, trước khi quên mất.

Những thuộc tính mới có thể được phát hiện ra từ:

- văn bản use case,
- các bản thiết kế giao diện người dùng (UI prototypes),
- hoặc từ các yêu cầu chức năng.

Lưu ý: Cố gắng đừng để bị xao nhãng sang việc **gán các phương thức (operations) cho lớp** ở giai đoạn này — đó là hoạt động của **thiết kế chi tiết**. Nếu bạn dành thời gian cho việc đó ngay bây giờ, rất có thể bạn sẽ phải **làm lại từ đầu sau này**. Chu trình phản hồi giữa **mô hình robustness** và **mô hình tĩnh (static model)** được minh họa trong Hình 5-18.

CHECK:

**Have you covered all of your alternate courses?
identified all of the methods/functions?
mapped all data flows between entities?**

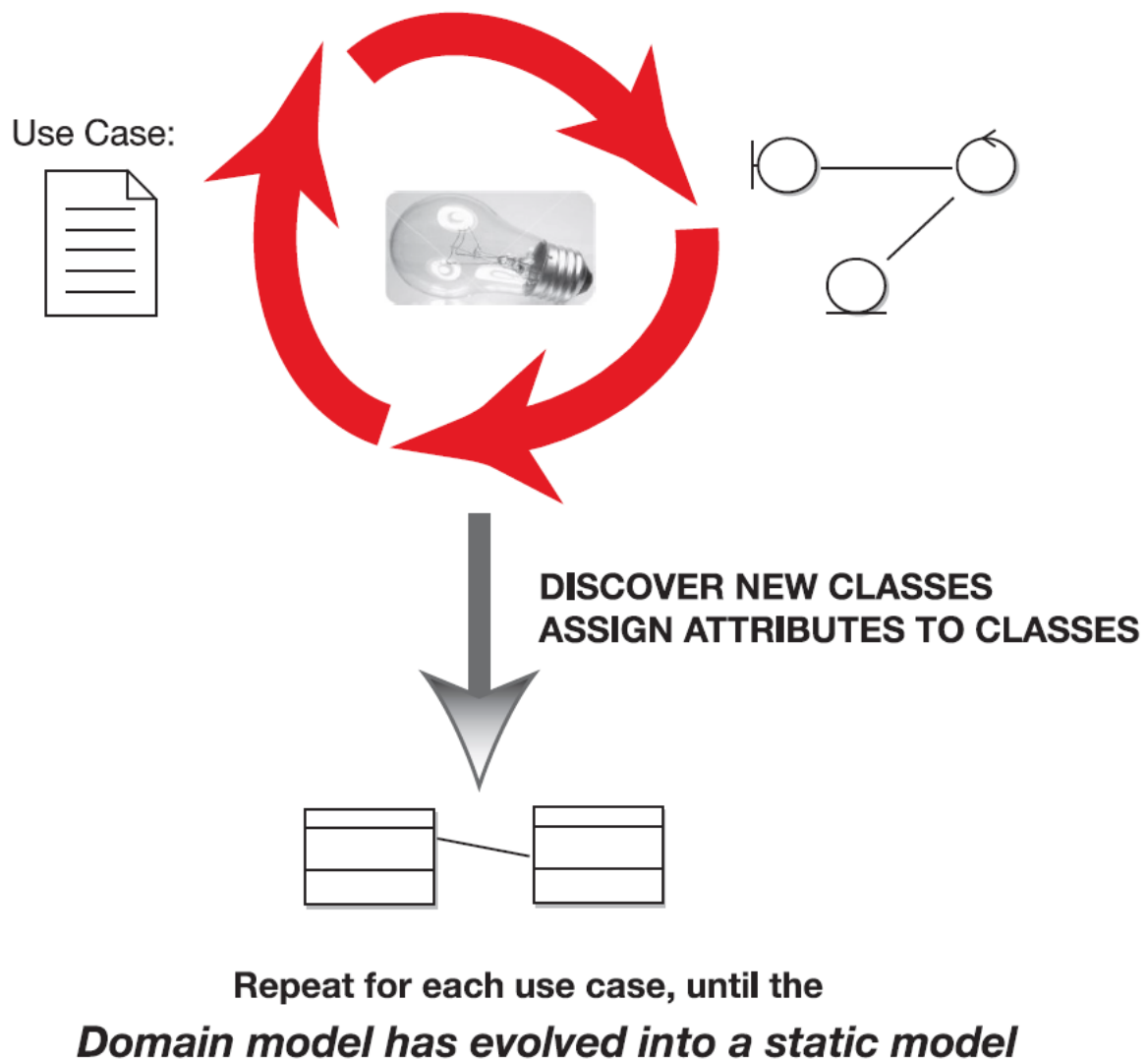


Figure 5-18. *Robustness model/static model feedback loop*

Hình 5-19 cho thấy mô hình miền (static model) của hệ thống Internet Bookstore sau khi thực hiện **robustness analysis** cho hai use case:

- *Show Book Details*
- *Write Customer Review*

Những lớp hoặc thuộc tính được bổ sung hoặc cập nhật nhờ vào quá trình robustness analysis được tô màu đỏ trong sơ đồ.

Các thay đổi được thực hiện bao gồm:

- Sau khi thực hiện robustness analysis cho use case *Show Book Details* (xem Hình 5-11):
 - Đã bổ sung các thuộc tính title và synopsis vào lớp Book.
(Các chi tiết này không được nhắc đến trong văn bản use case, nhưng được tìm thấy khi xem bản thiết kế UI.)
- Sau khi thực hiện robustness analysis cho use case *Write Customer Review* (xem Hình 5-17):
 - Đã thêm lớp CustomerSession, gán cho nó một thuộc tính loggedIn (kiểu Boolean), và liên kết nó với CustomerAccount (lớp này sau này, ở Chương 8, sẽ được đổi thành Customer).
 - Đã xóa lớp CustomerRating, vì nó đã được chuyển thành một thuộc tính của BookReview.
 - Đã thêm một số thuộc tính mới cho BookReview, được nhắc tới trong văn bản use case.

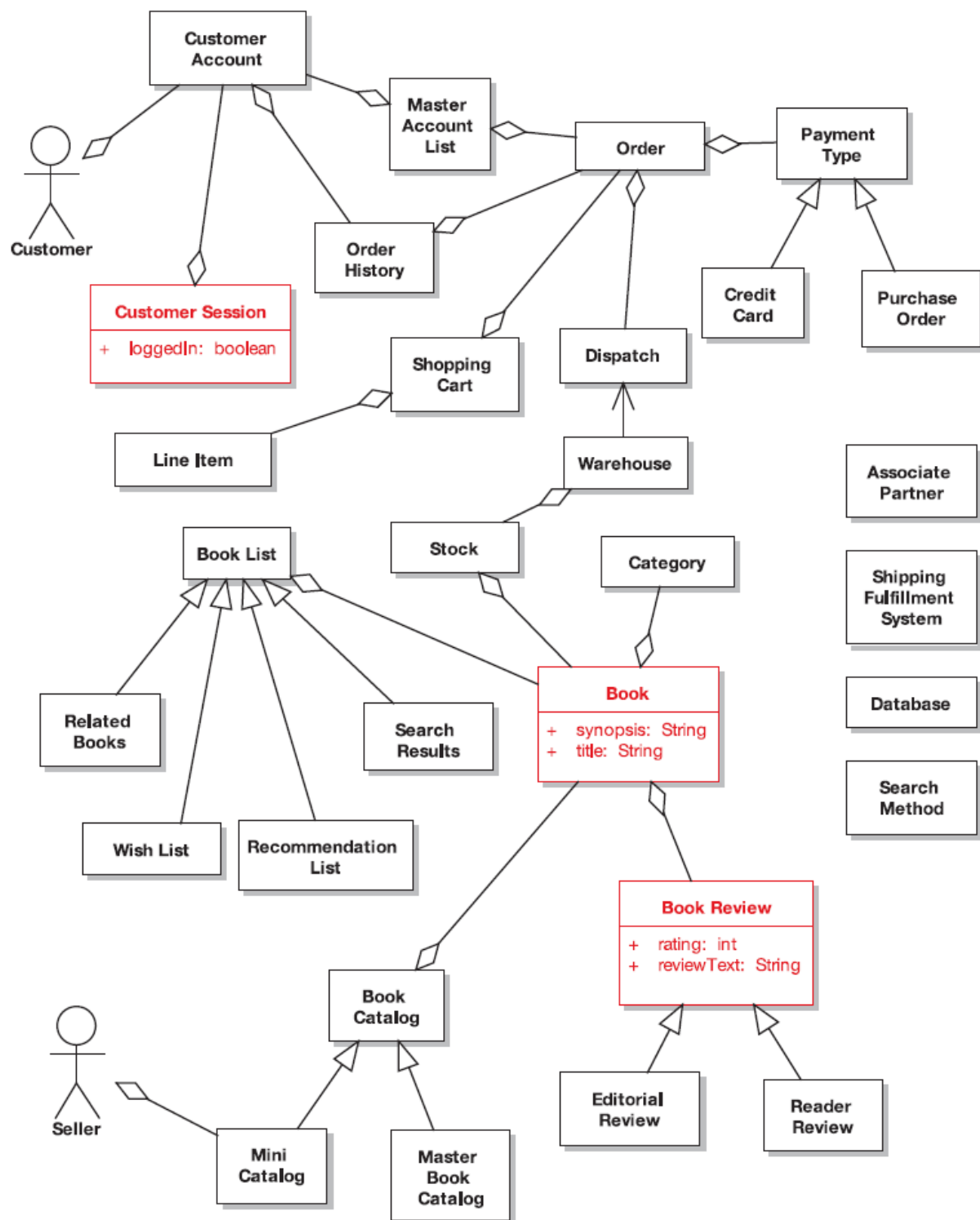


Figure 5-19. Static model for the Internet Bookstore, after robustness analysis for two use cases

Có một bước cuối cùng trước khi bạn hoàn thành robustness analysis: Sau khi đã vẽ tất cả các sơ đồ robustness cho toàn bộ release, bạn phải quét lại toàn bộ một lượt, và đảm bảo rằng mọi chi tiết liên quan đều đã được phản ánh ngược lại vào mô hình

miền. Đây là bước chốt vòng phản hồi phân tích – thiết kế sơ bộ, nhằm đảm bảo mô hình miền và các sơ đồ robustness luôn nhất quán với nhau.

Tổng kết

Bài giảng 5 đã đưa các bạn đi qua toàn bộ nội dung của Chapter 5 – Robustness Analysis: từ bối cảnh thực tế, vị trí trong quy trình ICONIX, khái niệm và quy tắc, phương pháp thực hiện, các nguyên tắc vàng, ví dụ Internet Bookstore minh họa, đến các lỗi thường gặp và tư tưởng cốt lõi. Đây là một trong những bài quan trọng nhất của khóa học, vì nó quyết định chất lượng thiết kế và khả năng kiểm soát yêu cầu của toàn dự án.

Câu hỏi ôn tập

1. Trình bày bối cảnh ra đời của Robustness Analysis và vấn đề mà kỹ thuật này giải quyết.
2. Nêu vị trí của Robustness Analysis trong ICONIX Process và ý nghĩa của PDR (Preliminary Design Review).
3. Trình bày cấu trúc của sơ đồ robustness. Ba loại đối tượng là gì? Quy tắc noun–verb–noun ra sao?
4. Mô tả quy trình 6 bước thực hiện Robustness Analysis cho một use case.
5. Nêu 5 trong số Top 10 Guidelines và giải thích vì sao chúng quan trọng.
6. Trong ví dụ “Login”, hãy liệt kê các boundary, controller, entity và alternate flow.
7. Trong ví dụ “Show Book Details”, lỗi phổ biến nào được phát hiện nhờ robustness diagram?
8. Trong ví dụ “Write Customer Review”, thực thể nào được phát hiện thêm trong quá trình robustness?
9. Nêu 3 lỗi thường gặp khi sinh viên vẽ sơ đồ robustness và cách khắc phục.
10. Giải thích vì sao robustness diagram được gọi là “bản thiết kế sơ khai” của use case.